

Alphabete, Wörter und Sprachen

Alphabete

Endliche, nichtleere Menge von Symbolen

z.B. $\Sigma = \{a, b, c\}$, Menge der 3 Symbole a, b, c

$\Sigma = \{-, +, \cdot, : \}$, Menge der Symbole für die Grundrechenarten

$\mathbb{N}$ ist kein Alphabet (unendliche Mächtigkeiten)

Wörter

Endliche Folge von Symbolen eines Alphabetes z.B. 100111 ist ein Wort über dem Alphabet $\{0, 1\}$

Leeres Wort ϵ

Wort, das keine Symbole enthält

Wort über jedem Alphabet

Spiegelwort w^R

Wenn $w = w^R$, wird w als Palindrom bezeichnet

Es gilt $|w| = |w^R|$ und $|w|_x = |w^R|_x$

z.B. $(abc)^R = cba$

Länge eines Wortes $|w|$

Anzahl Symbole einer Folge

z.B. $|abcd| = 4$

$|\epsilon| = 0$

$|\text{Hello World}| = 11$ ① Leerzeichen sind auch Symbole

Absolute Häufigkeit $|w|_x$

Häufigkeit eines Symbols x in einem Wort w

z.B. $|100111|_1 = 4$

Operationen über Wörter

Kleinsche Hülle Σ^*

Menge aller Wörter, welche man mit einem Alphabet bilden kann

$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$

z.B. $\{0, 1\}$ ist $\Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, \dots\}$

Konkatenation

x und y zwei beliebige Wörter, dann ist $x \circ y = xy$

z.B. $x = 01001$ und $y = 110$, $xy = 01001110$

Teilwort (Infix)

Echtes Teilwort, wenn nicht identisch mit w

z.B. $\epsilon, a, b, ab, abb, bb, abba, bba, ba$ sind Teilwörter von $\overbrace{abba}^w$

$abba$ ist kein echtes Teilwort von $abba$ (alle anderen schon)

Präfix

Echtes Präfix, wenn nicht identisch mit w

z.B. $\epsilon, a, ab, abb, \overbrace{abba}^w$ sind Präfixe von $abba$

$abba$ ist kein echtes Präfix von $abba$ (alle anderen schon)

Suffix

Echtes Suffix, wenn nicht identisch mit w

z.B. $\overbrace{abba}^w, bba, ba, a, \epsilon$ sind Suffixe von $abba$

$abba$ ist kein echtes Suffix von $abba$ (alle anderen schon)

Menge aller Wörter

Menge aller Wörter der Länge k : Σ^k

z.B. $\Sigma = \{a, b, c\}$, $\Sigma^2 = \{aa, ab, ac, ba, bb, bc, ca, cb, cc\}$ ($3^2 = 9$)

① Es gilt immer $\Sigma^0 = \{\epsilon\}$ unabhängig von Σ

Positive Hülle Σ^+

Menge aller nichtleeren Wörter

$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$

z.B. $\{0, 1\}$ ist $\Sigma^+ = \{0, 1, 00, 01, 10, 11, 000, \dots\}$

Wortpotenzen

Man kann Wörter kürzer darstellen. $x^0 = \epsilon$, $x^{n+1} = x^n \circ x = x^n x$

z.B. $a^3 = a^2 a = a^1 a a = a^0 a a a = a a a$

$bbababababbbaaaabab = b^2 (ab)^4 b a^4 b a b$

Sprachen

Teilmenge $L \subseteq \Sigma^*$ von Wörtern über einem Alphabet Σ

Sprachen können aus unendlich vielen Wörtern bestehen

Σ^* ist eine Sprache über jedem Alphabet Σ

$\{\} = \emptyset$ beschreibt die leere Sprache

$\{\epsilon\}$ ist die Sprache, die aus dem leeren Wort ϵ besteht für jedes Alphabet Σ

z.B. $L = \{\epsilon, 10, 1100, 111000, \dots\}$

Konkatenation

Sprache AB besteht aus Wörtern, die man ohne Überschneiden in ein Präfix aus A und ein Suffix aus B aufteilen kann

AB ist eine Sprache über dem Alphabet $\Sigma \cup T$

z.B. Sprachen A und B wie folgt:

A : alle Binärwörter, die mit 1 beginnen

B : alle Binärwörter, die mit 0 enden

ϵ ✗, 10 ✓, 01010 ✗, 1010 ✓, 11 ✗, 1100110010 ✓

Kleensche Hülle

A^* einer Sprache A ist durch $\{\epsilon\} \cup A \cup AA \cup AAA \cup \dots$ definiert

z.B. A^* von $A = \{aa, ab, ba, bb\}$

ϵ ✓, ababa ✗, abbaba ✓, abaaabb ✗, abbaabba ✓, aaaaaa ✗

① Sprache gerade also müssen auch Wörter gerade sein, ϵ gehört immer dazu

Entscheidungsprobleme

Entscheiden ob ein Wort zu einer Sprache gehört oder nicht

z.B. Alphabet $\Sigma = \{0,1\}$, Sprache $L_p = \{w \mid w \text{ ist eine Primzahl}\}$, Wörter x aus Σ^*

① Alle Wörter welche mit 0 enden, stellen eine gerade Zahl dar, also keine Primzahl (ausser 2)

Für andere Wörter ist diese Entscheidung erheblich aufwendiger

Komplexitätstheorie

Komplexitätsmasse

- Zeitkomplexität: Laufzeit des besten Programms, welches das Problem löst
- Platzkomplexität: Speicherplatzbedarf des besten Programms
- Beschreibungskomplexität: Länge des kürzesten Programms

Laufzeit

Haltende TM mit Eingabealphabet Σ . Zeitbedarf von M auf Eingabe $w \in \Sigma$ ist:

- Anzahl Konfigurationsübergänge von Berechnung der TM auf w

① Zeitbedarf von TM auf Eingabe der Länge n im schlechtesten Fall = n

Asymptotische Komplexitätsmessung

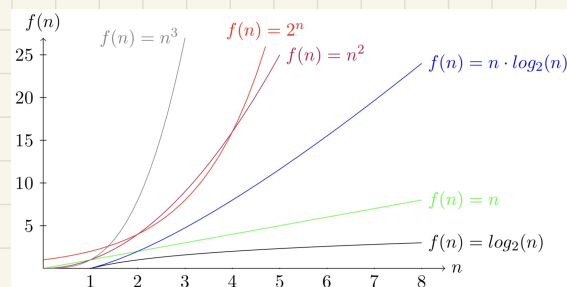
- Methode zur Bewertung der Effizienz von Algorithmen
- Analysiert Veränderung von Laufzeit / Speicherbedarf von Algorithmen, bei zunehmender Eingabegrösse
- Konstante Faktoren und niedrige Potenzen der Eingabegrösse werden vernachlässigt

Asymptotisches Wachstum

Wenn Funktion $f(x)$ asymptotisch schneller wächst als Funktion $g(x)$, bedeutet dies, dass $f(x)$ im Vergleich zu $g(x)$ für grössere x -Werte schneller wächst

O-Notation

- $f \in O(g)$:
- f wächst asymptotisch nicht schneller als g
 - n_0 und c existiert, sodass für alle $n \geq n_0$ gilt: $f(n) \leq c \cdot g(n)$



Rechenregeln:

- Konstante Vorfaktoren ignorieren: $c \cdot f(n) \in O(f(n))$
- Für eine Konstante c gilt: $c \in O(1)$ (Konstanter Aufwand)
- Bei Polynomen ist nur die höchste Potenz entscheidend: $a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0 \in O(n^k)$
- Transitiv: $f(n) \in O(g(n))$ und $g(n) \in O(h(n))$ folgt $f(n) \in O(h(n))$

z.B.

$O(n)$	$7n + 4$ (Linearer Aufwand)
$O(n^3)$	$25n^2 + n^3 + 100n$ (Polynomial Aufwand)
$O(n^2 \log(n))$	$n^2 + n \cdot n \cdot (\log(n)) + 20n^2 + 50n + 100$ (Logarithmischer Aufwand)
$O(2^n)$	$10^{20} + 3n^3 + 2^n + 2^{10} + 2^{30}$ (Exponentieller Aufwand)

Omega-Notation

- $f \in \Omega(g)$:
- f wächst asymptotisch mind. so schnell wie g
 - n_0 und d existieren, sodass für alle $n \geq n_0$ gilt: $f(n) \geq \frac{1}{d} \cdot g(n)$

Theta-Notation

- $f \in \Theta(g)$:
- f und g sind asymptotisch gleich
 - $f(n) \in O(g(n))$ und $f(n) \in \Omega(g(n))$

Obere Schranke (O-Notation)

- Obergrenze für Komplexität
- Maximaler Aufwand an Zeit und Operationen, welcher für Lösung des Problems erforderlich ist

Wenn Algorithmus mit O-Notation $O(n^2)$ angegeben wird, bedeutet dies, dass die Zeitkomplexität **quadratisch** zur Grösse der Eingabe wächst und der Algorithmus für grössere Eingaben langsam wird.

Untere Schranke (Omega-Notation)

- Untergrenze für Komplexität
- Minimaler Aufwand an Zeit und Operationen, welcher für Lösung des Problems erforderlich ist

Wenn Algorithmus mit der Omega-Notation $\Omega(\log(n))$ angegeben wird, bedeutet dies, dass jeder effiziente Algorithmus mind. eine Laufzeit von **$\log(n)$** aufweisen muss.

① Schwierig, weil gezeigt werden muss, dass alle Algorithmen keine bessere Schranke liefern

Polynomzeit

- Laufzeit eines Algorithmus
- Lösbar wenn es eine obere Schranke $O(n^c)$ gibt für Konstante $c \geq 1$
- Laufzeit wächst nicht exponentiell mit der Eingabegrösse
- Laufzeit ist durch Polynom der Eingabegrösse beschränkt

Polynomzeit-Verifizierer

- Prüfen ob vorgeschlagene Lösung für ein Problem korrekt ist
Verifizierer arbeitet in polynomieller Zeit $\rightarrow$ Laufzeit Verifikation ist auf Grösse des Problems beschränkt
- Kann keine Lösung finden nur verifizieren

① Annahme $P \neq NP$: Nicht alle Probleme, welche in Polynomzeit lösbar sind, können in Polynomzeit verifiziert werden

Klassifizierung von Problemen

P:

- Klasse aller Sprachen, bei welchen eine Lösung in Polynomzeit gefunden werden kann

NP (Nichtdeterministisch Polynomiell):

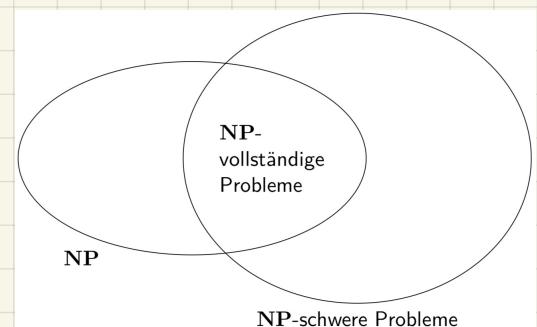
- Klasse aller Sprachen, bei welchen eine Lösung in Polynomzeit verifiziert werden kann
- NP ist Menge aller Sprachen für die ein Polynomzeit-Verifizierer existiert

NP-Schwer:

- Wenn genau so schwierig wie schwierigstes Problem von NP
- Hinweis darauf, dass Problem komplex ist und ineffiziente Lösung hat
- Lässt sich auf NP Probleme reduzieren

NP-vollständig:

- Problem welches NP-Schwer und NP ist
- Wenn P_1 NP-Schwer und P_2 in NP ist und $P_1 \leq_p P_2$, ist P_2 NP-vollständig



Polynomiell Reduzierbar

$L_1 \leq_p L_2$: L_1 ist polynomiell auf L_2 reduzierbar

- L_2 ist mind. so schwer lösbar wie L_1

SAT

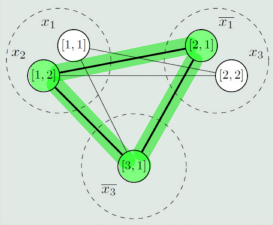
- NP-vollständiges Problem
- Entscheidet ob eine Formel in KNF erfüllbar ist $(x_1 \vee x_2 \vee x_4) \wedge (x_3 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_3)$

CLIQUE

- NP-vollständiges Problem
- Gruppe von Knoten finden, in der jeder Knoten mit jedem anderen verbunden ist
- Besteht aus Knoten, die durch Kanten miteinander verbunden sein können
- **Grösse von CLIQUE** = Anzahl verbundener Knoten

Beispiel (CLIQUE ist NP-vollständig)

Beispiel für die Konstruktion: $\varphi = (x_1 \vee x_2) \wedge (\bar{x}_1 \vee x_3) \wedge (\bar{x}_3)$



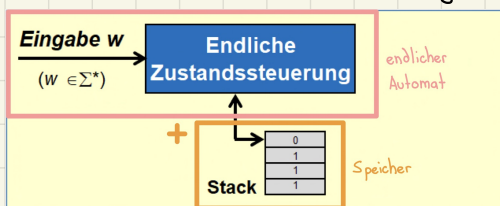
Die Formel ist erfüllbar.

$\Leftrightarrow$

Der Graph enthält eine Clique der **Grösse 3**.

Kellerautomaten

Automatenmodell für die Erkennung von kontextfreien Sprachen
Endlicher Automat mit zusätzlichem (unbegrenztem) Stack



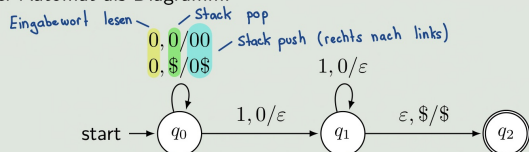
Beispiel (Informell)

Ein Kellerautomat für die kontextfreie Sprache $\{0^n 1^n \mid n > 0\}$:

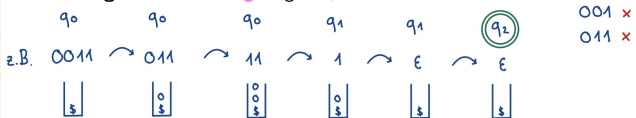
- Solange keine Eins gelesen wird, lege die gelesenen Nullen auf dem Keller ab. Sobald Einsen gelesen werden, entferne für jede gelesene Eins eine Null vom Keller.
- Akzeptiere das Eingabewort, wenn die Berechnung im akzeptierenden Zustand endet. Der akzeptierende Zustand wird erreicht, wenn (der Keller leer ist und) das ganze Wort gelesen wurde.
- Andernfalls verwirfe die Eingabe.

Beispiel (Grafisch)

Der Automat als Diagramm.



Anmerkung: Das Zeichen \$ zeigt an, dass der Stack leer ist.



Berechnungsschritt

$\delta(q, b, c) = (p, w)$

● aktueller Zustand

● gelesenes Symbol

● von Stack gelesen (pop)

● neuer Zustand

● alle Zeichen die auf den Stack geschrieben werden (push)



Berechnung

Akzeptierend wenn für Endkonfiguration (q_f, ϵ, γ) gilt, dass $q_f \in F$

z.B. $(q_0, a, xy... \$) \vdash (q_1, \epsilon, axy... \$)$

Äquivalenz mit kontextfreien Sprachen

Sprache kontextfrei wenn NKA gibt der diese erkennt.

NKA ist gleichmächtig wie kontextfreie Grammatik

Nicht kontextfreie Sprachen:

Beispiel

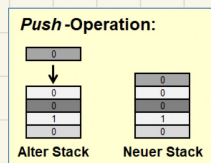
Die Sprache $L = \{0^n 1^n 2^n \mid n > 0\}$ ist nicht kontextfrei.

- Ein Kellerautomat, der ein Wort aus L akzeptieren würde, müsste sich die Anzahl der eingelesenen 0 und 1 merken.
- Mit dem Keller kann aber nur **einmal** eine Anzahl verglichen werden, danach sind die **Symbole nicht mehr auf dem Keller**.
- Mit Zuständen kann, wie beim EA, nicht beliebig gezählt werden.

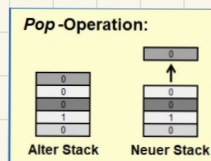
Keller (Stapel, Stack)

Element an oberster Stelle hinzufügen (push) / entfernen (pop)

⚠ First-in / Last-out, \$ = Stack ist leer



beliebig viele Zeichen pro Schritt



nur 1 Zeichen pro Schritt

Konfiguration

Definition (Konfiguration)

Sei $M = (Q, \Sigma, \Gamma, \delta, q_0, \$, F)$ ein NKA.

Eine **Konfiguration** von M ist ein Element (q, w, γ) aus $Q \times \Sigma^* \times \Gamma^*$, wobei

- q für den Zustand steht, Zustand + Restwort + Stack
- w die verbleibende Eingabe repräsentiert,
- γ für den Inhalt des Kellers steht. (Dabei steht das Symbol ganz links für das oberste Symbol.)

Mit $(q_0, w, \$)$ bezeichnen wir die **Startkonfiguration** für die Eingabe w und mit (q, ϵ, γ) eine **Endkonfiguration**.

Deterministischer Kellerautomat (KA)

Definition (deterministischer Kellerautomat)

Ein **deterministischer Kellerautomat (KA)** M ist ein 7-Tupel $(Q, \Sigma, \Gamma, \delta, q_0, \$, F)$, wobei

- Q ist eine endliche Menge von Zuständen.
- Σ ist das Alphabet der Eingabe.
- Γ ist das Alphabet des Kellers. Wörter
- $\delta : Q \times (\Sigma \cup \epsilon) \times \Gamma \rightarrow Q \times \Gamma^*$ ist eine (partielle) Übergangsfunktion.
- $q_0 \in Q$ ist der Startzustand.
- $\$ \in \Gamma$ ist ein ausgezeichnetes Symbol vom Alphabet des Kellers.
- $F \subseteq Q$ ist die Menge der akzeptierenden Zustände.

Anmerkung: Anfangs enthält der Keller eine Instanz des Symbols \$.

Es gilt, wenn $\delta(q, b, x)$ definiert ist, dann ist $\delta(q, \epsilon, x)$ undefiniert



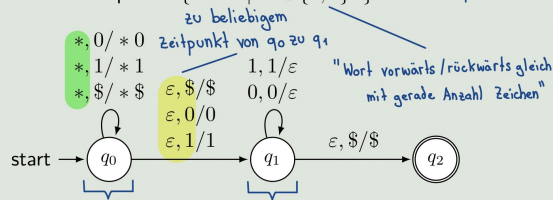
Nichtdeterministischer Kellerautomat (NKA)

Zusatzbedingung an die Übergangsfunktion fällt weg

$\delta : Q \times (\Sigma \cup \epsilon) \times \Gamma \rightarrow P(Q \times \Gamma^*)$ Unterschied zu KA

z.B. Beispiel

Kellerautomat für die Sprache $\{ww^R \mid w \in \{0, 1\}^*\}$: z.B. 100|001



Anmerkung

- Der * steht im Diagramm für ein beliebiges Zeichen aus Σ (d. h. in diesem Beispiel 0 oder 1) und dient der Vereinfachung.

Kontextfreie Grammatiken

Kontextfreie Grammatik G (KFG) ist ein 4-Tupel (N, Σ, P, A) mit :

N : Alphabet der Nichtterminale (Variablen), z.B. $N = \{A\}$

Σ : Alphabet der Terminale, z.B. $\Sigma = \{0,1\}$

P : Endliche Mengen von Produktionen (Regeln)

Form jeder Produktion: $X \rightarrow \beta$, z.B. $P = \{A \rightarrow 0A1, A \rightarrow \epsilon\} / P = \{A \rightarrow 0A1 \mid \epsilon\}$

Kopf $X \in N$ / Rumpf / Satzform $\beta \in (N \cup \Sigma)^*$

A : Startsymbol, wobei $A \in N$

z.B. KFG für nicht-reguläre Sprache: $\{0^n 1^n \mid n \in \mathbb{N}\}$

① $A \rightarrow 0A1$, ② $A \rightarrow \varepsilon$

$$G = (N, \Sigma, P, A) = (\{A\}, \{0, 1\}, \{A \rightarrow 0A1, A \rightarrow \epsilon\}, A)$$

Ableitung des Wortes 000111 in der Grammatik: $A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000111$

Ableitungsschritt

Satzform $\alpha A \beta$ wird durch Produktion in Satzform $\alpha \gamma \beta$ abgeleitet.

$$\alpha A \beta \Rightarrow \alpha \gamma \beta$$

Ableitung

Folge von Ableitungsschritten, wodurch aus Satzform α das Wort w abgeleitet wird.

$$\alpha \Rightarrow \dots \Rightarrow w$$

Wort $w \in \Sigma^*$ in KFG ableitbar, wenn es Ableitung in KFG gibt, die mit Startsymbol A beginnt und mit Wort w endet.

$A \xRightarrow{*} w \text{ (1) } w \text{ wird von } A \text{ erzeugt / generiert}$

Linksseitige- und rechtsseitige Ableitung

Linksseitige Ableitung: Am weitesten links stehende Variable wird abgeleitet

z.B. $A \rightarrow AA \mid (A) \mid \epsilon$, $A \Rightarrow AA \Rightarrow (A)A \Rightarrow ((A))A \Rightarrow (((A)))A \Rightarrow (((A)))((A)) \Rightarrow (((A)))((A))((A))$

Rechtsseitige Ableitung: Am weitesten rechts stehende Variable wird abgeleitet

z.B. $A \rightarrow AA \mid (A) \mid \epsilon$, $A \Rightarrow AA \Rightarrow A(A) \Rightarrow A() \Rightarrow (A)() \Rightarrow ((A))() \Rightarrow (((A)))()$

Sprache einer kontextfreien Grammatik

G erzeugte Sprache $L(G)$ beinhaltet alle Wörter, die in G aus Startsymbol A ableitbar sind.

$$L(G) = \{ w \in \Sigma^* \mid A \Rightarrow^* w \}$$

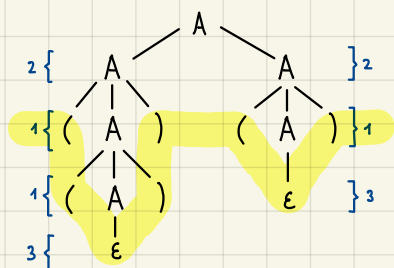
Kontextfreie Sprache

Wenn es für Sprache L eine kontextfreie Grammatik G mit $L = L(G)$ gibt.

Ableitungsbaum / Parsebaum

Graphische Darstellung einer Ableitung

z.B. $A \rightarrow \underbrace{(A)}_1 \mid \underbrace{AA}_2 \mid \underbrace{\epsilon}_3$ für Wort $(()) ()$



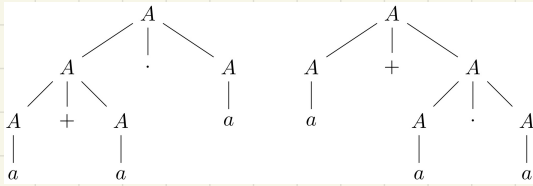
Mehrdeutigkeit

Wort besitzt mehrere Ableitungsbäume

Mehrdeutigkeit von KFG eliminieren:

- Korrekte Klammerung erzwingen z.B. $a+(a \cdot a)$ statt $a+a \cdot a$
- Grammatik anpassen z.B. Terme, Faktoren
- Produktionen einen Vorrang geben z.B. $A \rightarrow A+A$ Vorrang über $A \rightarrow A \cdot A$

z.B. $G = (\{A\}, \{a, \cdot, +, ()\}, P, A)$, $P = \{A \rightarrow A \mid A \cdot A \mid (A) \mid a\}$ ⚠ $a = \text{beliebige Binärzahl}$



Linksseitige Ableitungen für $a+a \cdot a$:

$A \Rightarrow A+A \Rightarrow a+A \Rightarrow a+A \cdot A \Rightarrow a+a \cdot A \Rightarrow a+a \cdot a$
 $A \Rightarrow A \cdot A \Rightarrow A+A \cdot A \Rightarrow a+A \cdot A \Rightarrow a+a \cdot A \Rightarrow a+a \cdot a$

} mehrdeutig
 $a+(a \cdot a) \neq (a+a) \cdot a$

Inhärent Mehrdeutig

Kontextfreie Sprache, für die alle Grammatiken mehrdeutig sind

z.B. $L = \{1^i 2^j 3^k \mid i=j \text{ oder } j=k, \text{ mit } i, j, k \in \mathbb{N}\}$

$1112223 \in L$

⚠ $L = \{1^i 2^j 3^k \mid i=j\} \cup \{1^i 2^j 3^k \mid j=k\}$

Wenn gleiche Anzahl 1,2,3, dann nicht klar welcher Ableitungsbaum

KFG für eine reguläre Sprache

Jede reguläre Sprache kann durch eine kontextfreie Grammatik beschrieben werden

z.B.

$L_5 = \{w \in \{0,1\}^* \mid |w|_1 \bmod 3 = 0\}$

Nichtterminale:

Q_0, Q_1, Q_2

Produktionen:

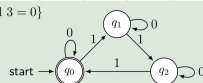
$Q_0 \rightarrow 0Q_0 \mid 1Q_1 \mid \varepsilon$

$Q_1 \rightarrow 0Q_1 \mid 1Q_2$

$Q_2 \rightarrow 0Q_2 \mid 1Q_0$

Beispiel für Ableitung von $w = 10011$:

$Q_0 \Rightarrow 1Q_1 \Rightarrow 10Q_1 \Rightarrow 100Q_1 \Rightarrow 1001Q_2 \Rightarrow 10011Q_0 \Rightarrow 10011$



Entscheidbarkeit

Entscheidbar

Sprache $A \subseteq \Sigma^*$ entscheidbar, wenn TM sich wie folgt verhält:

- TM hält nach endlich vielen Schritten
- Bandinhalt $x \in A$: TM hält mit Bandinhalt «1» (Ja) an
- Bandinhalt $x \in \Sigma^* \setminus A$: TM hält mit Bandinhalt «0» (Nein) an

Äquivalente Aussagen:

- Jede entscheidbare Sprache ist auch semi-entscheidbar
- TM existiert, die das Entscheidungsproblem $T(\Sigma, A)$ löst
- WHILE-Programm existiert, dass bei zu A gehörendem Wort immer terminiert (Entscheidungsverfahren)

Semi-entscheidbar

Sprache $A \subseteq \Sigma^*$ semi-entscheidbar, wenn TM sich wie folgt verhält:

- TM hält nur bei validem Bandinhalt an
- Bandinhalt $x \in A$: TM hält mit Bandinhalt «1» (Ja) an
- Bandinhalt $x \in \Sigma^* \setminus A$: TM hält nie an

Äquivalente Aussagen:

- A ist rekursiv aufzählbar
- A ist total berechenbare Funktion
- WHILE-Programm existiert, dass bei zu A gehörendem Wort immer terminiert
- WHILE-Programm existiert, dass bei nicht zu A gehörendem Wort nie terminiert

① A genau dann entscheidbar, wenn sowohl A als auch $\bar{A}$ semi-entscheidbar sind

Reduktion

Schwieriges Problem auf ein bereits bekanntes Problem zurückführen, um Lösbarkeit zu bestimmen:

- Wenn Problem B unlösbar ist, dann ist es auch Problem A
- Wenn Problem B lösbar ist, gibt es eine Möglichkeit Problem A zu lösen
- $A \leq B$ (A ist reduzierbar auf B , B kann mind. gleichviel wie A)
- Transitivität: $A \leq B$ und $B \leq C \rightarrow A \leq C$

Halteproblem

- Allgemeines Halteproblem H : Hält TM auf gegebenem Input an?
- Leeres Halteproblem H_0 : Hält TM auf leerem Band an?
- Spezielles Halteproblem H_s : Hält TM auf Band welches die gleiche TM beinhaltet an?

Alle sind nicht entscheidbar (es gibt keine TM, die Entscheidung in endlicher Zeit treffen kann)

Probleme H , H_0 , H_s sind semi-entscheidbar

Endliche Automaten

Berechnung: Automat akzeptiert alle Wörter, die bei einer Berechnung in einem akzeptierenden Zustand enden.

Speicher : Kein Speicher und keine Variablen

$L(M)$: Menge aller Wörter die von M akzeptiert werden

Bausteine $(M = (Q, \Sigma, \delta, q_0, F))$

Zustände:

$$Q = \{q_0, q_1, \dots, q_n\} \quad (n \in \mathbb{N})$$



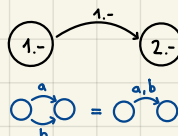
Eingabealphabet:

$$\Sigma = \{a_1, a_2, a_m\} \quad (m \in \mathbb{N})$$

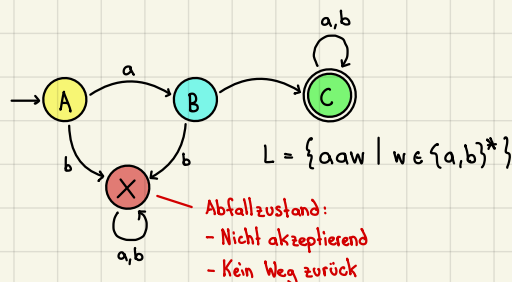
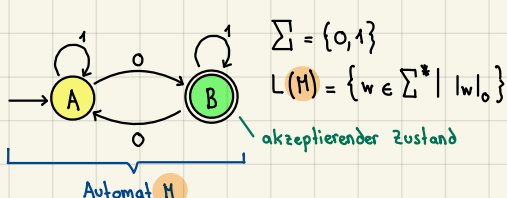
0.-, 1.-, 2.-
Eingabewort von links nach rechts

Übergangsfunktionen:

$$\delta : Q \times \Sigma \rightarrow Q$$



Beispiele:



A: 1011101

A: 011101

B: 11101

B: 1101

B: 101

B: 01

A: 1

A: ϵ

A: aaba

B: aba

C: ba

C: a

C: ϵ ✓

A: abab

B: bab

X: ab

X: b

X: ϵ ✗

A: aaaa

B: aaa

C: aa

C: a

C: ϵ ✓

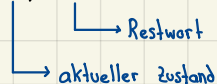
Konfigurationen

Momentaufnahme einer Berechnung, die alle Informationen um weiter zu arbeiten.

Vergangenheit ist egal, nur aktueller Zustand ist wichtig.

Konfiguration $M = \text{Element aus } Q \times \Sigma^*$

z.B. $(A, abba)$



Startkonfiguration : $(q_0, w) \in \{q_0\} \times \Sigma^*$! Kann während Berechnung mehrmals vorkommen

z.B. $w = a_1, a_2, a_2, a_1 \rightarrow (q_0, a_1, a_2, a_2, a_1)$

Endkonfiguration : $Q \times \{\epsilon\}$! Kommt genau 1-Mal vor

z.B. $M: (q_0, \epsilon), (q_1, \epsilon), (q_2, \epsilon), (q_3, \epsilon), (q_4, \epsilon)$

Berechnungsschritt : Anwendung der Übertragungsfunktion auf die aktuelle Konfiguration
(current state, noch zu lesen) $\vdash M$ (next state, suffix)

! Es gibt für jede Konfiguration einen Folgezustand

Berechnung : Endliche Folge von Berechnungsschritten

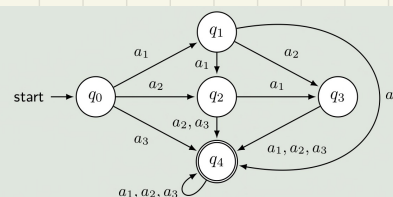
$(q_a, w_1 w_2 \dots w_n) \vdash M (q_b, w_2 \dots w_n) \vdash M \dots \vdash M (q_e, w_j \dots w_n)$

! Abkürzung: $(q_a, w_1 w_2 \dots w_n) \vdash^* M (q_e, w_j \dots w_n)$

Berechnung M auf einer Eingabe $w \in \Sigma^*$ ist akzeptierend wenn

$q_e \in F$ und verwerfend wenn $q_e \notin F$

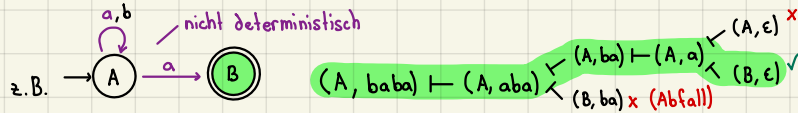
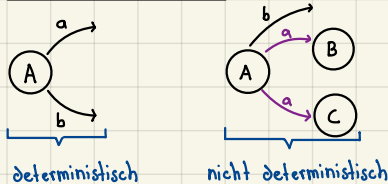
A:



Beispiele für Berechnungen von A ausgehend von q_1 bzw. q_0 mit je zwei Berechnungsschritten:

- $(q_1, a_2 a_2 a_1) \vdash_A (q_3, a_2 a_1) \vdash_A (q_4, a_1) \vdash_A (q_4, \epsilon)$
- $(q_0, a_1 a_2) \vdash_A (q_1, a_2) \vdash_A (q_3, \epsilon)$

Nicht-/Determinismus



! Wort wird akzeptiert sobald mind. 1 Pfad akzeptiert wird.

Nichtdeterministischer endlicher Automat (NEA)

$L(M)$ = Menge aller von M akzeptierten Wörter

$\delta: Q \times \Sigma \rightarrow P(Q)$ = Menge von Zuständen

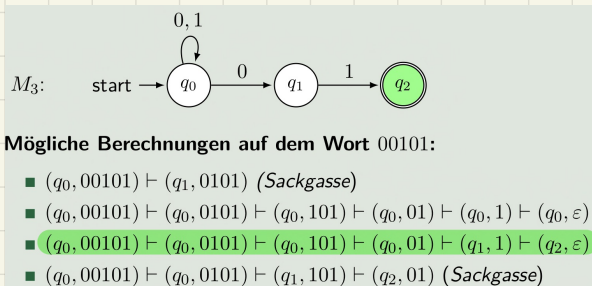
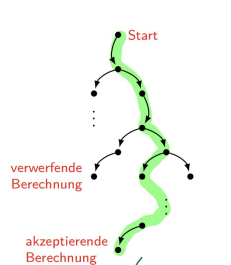
↳ Potenzmenge von Q = Menge aller Teilmengen von Q

Berechnung: Akzeptierend wenn mind. 1 akzeptierender Berechnungspfad existiert

Deterministische Berechnung:



Nichtdeterministische Berechnung:



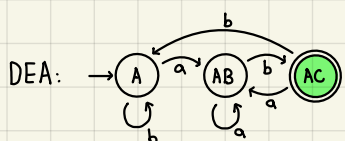
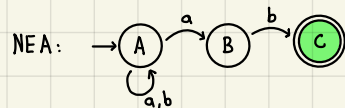
Äquivalenz von NEA und DEA

Jeder DEA ist ein NEA und ein ϵ -NEA.

Teilmengekonstruktion

Aus NEA einen DEA erstellen.

z.B. $L(M) = \{wab \mid w \in \{a,b\}^*\}$

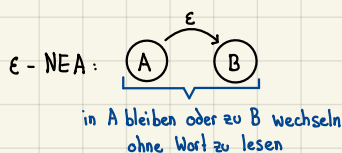
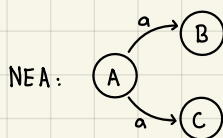


NEA mit ϵ -Übergängen

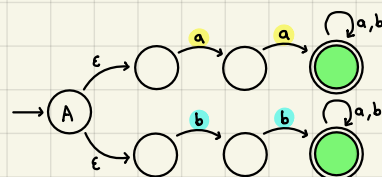
Spontan Zustand wechseln ohne Eingabesymbol zu lesen

$M = (Q, \Sigma, \delta, q_0, F)$

Übergangsfunktion $\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow P(Q)$



z.B. $L = \{aaw \mid w \in \{a,b\}^*\} \cup \{bbw \mid w \in \{a,b\}^*\}$



Äquivalenz von DEA und RA

Klasse der regulären Sprachen = Alle vom endlichen Automaten akzeptierte Sprachen

1. Fall : $RA = \emptyset$ ($L(RA) = \emptyset$)

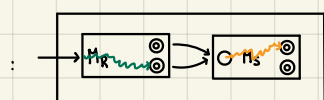
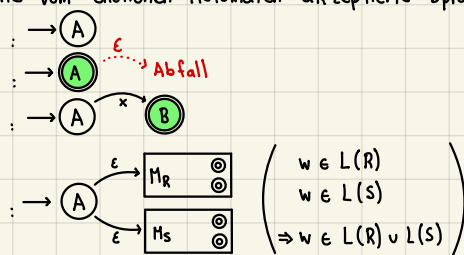
2. Fall : $RA = \epsilon$ ($L(RA) = \epsilon$)

3. Fall : $RA = x \in \Sigma$ ($L(RA) = \{x\}$)

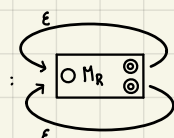
4. Fall : $RA = RS$

5. Fall : $RA = RS$

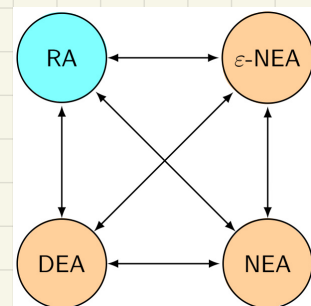
6. Fall : $RA = R^*$



$w \in L(RS), w \hat{=} \begin{matrix} \text{L(R)} & \text{L(S)} \end{matrix}$



! Akzeptierende Zustände von M_{RS} sind die akzeptierende Zustände von M_S



(light blue circle) Beschreibender Mechanismus
 (orange circle) Akzeptierender Mechanismus

Abschlusseigenschaften regulärer Sprachen

L_1, L_2 = reguläre Sprachen über Σ

Vereinigung : $L_1 \cup L_2 = \{w \mid w \in L_1 \text{ oder } w \in L_2\}$
 $\downarrow \quad \downarrow \quad \downarrow$
 $R \quad S \quad R \cup S$

Konkatenation : $L_1 \cdot L_2 = L_1 L_2 = \{w = w_1 w_2 \mid w_1 \in L_1 \text{ und } w_2 \in L_2\}$
 $\downarrow \quad \downarrow \quad \downarrow$
 $R \quad S \quad RS$

Kleensche Hülle : $L^* = \{w = w_1 w_2 \dots w_n \mid w_i \in L \text{ für alle } i \in \{1, 2, \dots, n\}\}$
 $\downarrow \quad \downarrow$
 $R \quad R^*$

Komplement : $\bar{L} = \Sigma^* - L = \{w \in \Sigma^* \mid w \notin L\}$



Schnitt : $L_1 \cap L_2 = \{w \mid w \in L_1 \text{ und } w \in L_2\}$

Mengendifferenz : $L_1 - L_2 = L_1 \cap \bar{L}_2 = \{w \mid w \in L_1 \text{ und } w \notin L_2\}$

Zustandsklassen

Klasse $[p] = \{w \in \Sigma^* \mid M \text{ endet nach dem Lesen von } w \text{ in } p\}$

① Menge aller Wörter Σ^* wird von Zustandsklassen $[p_0]$ bis $[p_n]$ partitioniert.

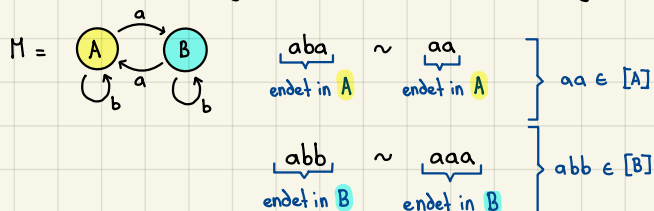
Eigenschaften: Jedes Wort landet in einem Zustand : $\Sigma^* = \bigcup_{p \in Q} [p]$

Kein Wort landet nach dem Lesen in zwei Zuständen : $[p] \cap [q] = \emptyset$, für alle $p \neq q, p, q \in Q$

Von M akzeptierte Sprache : $L(M) = \bigcup_{p \in F} [p]$

z.B. M sei ein DFA. Definiere Äquivalenzrelationen auf der Menge Σ^* (aller Wörter).

$w \sim v \Leftrightarrow M$ endet im gleichen Zustand wenn mit w oder v gestartet.



Feststellung: $w, v \in [q] \Rightarrow wx, vx$

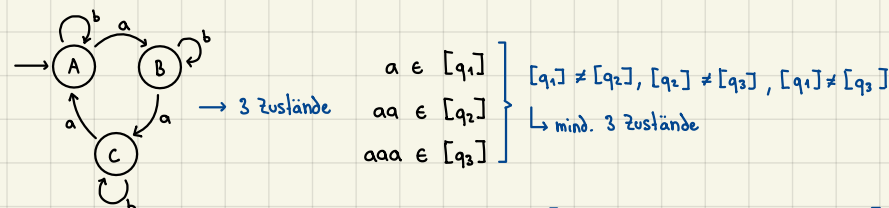
$M =$ } Enden im gleichem Zustand
 Unterscheidung zwischen w und v nicht mehr möglich

$w \sim v \Rightarrow wx \sim vx$

$w \sim v \Rightarrow (wx \in L(M) \Rightarrow vx \in L(M))$

Modulo

$L = \{w \in \{a, b\}^* \mid |w|_a = 0 \bmod 3\}$



$x=a$ $\left. \begin{array}{l} aa \notin L \\ aaa \in L \end{array} \right\} q_1 \neq q_2$ $\left. \begin{array}{l} aaa \in L \\ aaaa \notin L \end{array} \right\} q_1 \neq q_3$ $\left. \begin{array}{l} aaa \in L \\ aaaa \notin L \end{array} \right\} q_2 \neq q_3$

Nichtregularität

Unendlich viele Zustandsklassen $\Rightarrow$ unendlich viele Zustände $\Rightarrow$ kein EA/RA möglich

Mächtigkeit regulärer Sprachen

$L_8 = \{0^n 1^n \mid n \in \mathbb{N}\}$

Es gibt kein RA mit $L(R) = L_8$ und kein EA mit $L(M) = L_8$

z.B. $000111 : L(0^* 1^*) \Rightarrow 000111 \checkmark$

$0001 \otimes \notin L_8$

Grenzen endlicher Automaten

z.B. $\{a^n b^n \mid n \in \mathbb{N}\} \rightarrow$ nicht regulär

$$w_1 = a \in [q_1]$$

$$w_2 = aa \in [q_2]$$

$$w_3 = aaa \in [q_3]$$

$$w_4 = aaaa \in [q_4]$$

$$\vdots$$

$$w_n = a^n \in [q_n]$$

$$\vdots$$

$$\vdots$$

$$w_1 \in [q_1] \text{ und } w_2 \in [q_2] ?$$

$$\underbrace{ab}_{\in L}$$

$$\underbrace{aab}_{\notin L}$$

$$\Rightarrow q_1 \neq q_2$$

$$q_k = q_n \quad (n \neq k) ?$$

$$\underbrace{a^n b^n}_{\in L}$$

$$\underbrace{a^k b^n}_{\notin L}$$

$$\Rightarrow q_n \neq q_k$$

Es bräuchte unendlich viele Zustände

$\Rightarrow$ Es gibt keine endlichen Automaten mit unendlich vielen Zuständen

Berechnungsmodelle

Church-Turing-These

- Jedes algorithmisch lösbare Problem, kann von einer TM gelöst werden
- Computer und TM sind äquivalent

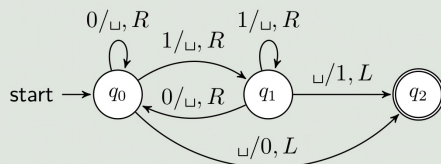
Turing-berechenbare Funktionen

Funktionen, die von einer TM berechnet werden können:

- TM liefert für Eingabe der Funktion korrekte Ausgabe
- TM hält in endlicher Zeit an

Beispiel (mod_2 Funktion)

Ist T durch das Übergangsdiagramm



gegeben, dann können wir T als Funktion

$$T: \mathbb{N} \rightarrow \mathbb{N} \text{ mit } T(n) = \text{mod}_2(n)$$

interpretieren. Die Funktion $f(n) = \text{mod}_2(n)$ ist Turing-berechenbar.

Turing-vollständigkeit

Fähigkeit jede Aufgabe zu lösen, welche eine TM lösen kann

Primitiv rekursive Funktionen

Menge von Funktionen, die aus Grundfunktionen konstruiert werden

❗ Entspricht LOOP-berechenbare Funktionen

Grundfunktionen:

Konstante Funktion: $C_k^n(x_1, \dots, x_n) = k$

- n = Anzahl Argumente, k = konstanter Rückgabewert
- z.B. $C_1^1(17) = 1$, $C_4^2(7, 2) = 4$

Nachfolgerfunktion: $\eta(x) = x + 1$

Projektion: $\pi_k^n(x_1, \dots, x_n) = x_k$

- $1 \leq k \leq n$
- z.B. $\pi_1^2(7, 5) = 7$, $\pi_1^1(x) = x$

Abschlusseigenschaften:

- $f(0, x) = h(x)$
- $f(k + 1, x) = g(f(k, x), k, x)$

z.B. Addition

$$\text{Add}(0, y) = y \quad \rightarrow \quad \text{Add}(0, y) = \pi_1^1(y)$$

$$\text{Add}(x + 1, y) = \text{Add}(x, y) + 1 \quad \rightarrow \quad \text{Add}(x + 1, y) = \eta(\pi_1^2(\text{Add}(x, y), x, y))$$

Ackermannfunktion

- Turing-vollständig
- Nicht LOOP berechenbar
- Nicht primitiv rekursiv

LOOP-Programme

- Variablen: $x_0, x_1, x_2, \dots$ (natürliche Zahlen ≥ 0 , initialisiert mit 0)
- Konstanten: 0, 1, 2, 3, 4, ...
- Trennzeichen: ;
- Zuweisung: = ($x_0 = \text{variable} + \text{Konstante}$, z.B. $x_3 = x_0 + 0$)
- Operationszeichen: + und - (Subtraktion grösseren von kleineren Wert immer 0, z.B. $9 - 2 = 0$)
- Schlüsselwörter: Loop, Do, End
- Terminiert immer (endlich viele Schritte)
- Nicht Turing-vollständig
- Primitiv rekursive Funktion

Beispiele:

Addition $\text{Add}(x, y) = x + y$

```
Loop x1 Do
  x2 = x2 + 1
End;
x0 = x2 + 0
```

Multiplikation $\text{Mul}(x, y) = x \cdot y$

```
Loop x2 Do
  Loop x1 Do
    x0 = x0 + 1
  End
End
```

Modulo $\text{Mod}(x, y)$

```
x3 = x1 + 1;
Loop x3 Do
  Loop x3 Do
    x0 = x3 + 0
  End;
  Loop x2 Do
    x3 = x3 - 1
  End
End;
x0 = x0 - 1
```

WHILE-Programme

Erweiterung von LOOP um While $x_i > 0$ Do ... End

- Jedes LOOP-Programm ist auch ein WHILE-Programm
- Terminiert nicht immer (kann unendlich lang laufen)
- Semantik von LOOP-Programm
- Turing-vollständig

Beispiele:

Multiplikation $\text{Mul}(x, y) = x \cdot y$

```
While x1 > 0 Do
  x1 = x1 - 1;
  Loop x2 Do
    x0 = x0 + 1
  End
End
```

GOTO-Programme

- Marker: M1, M2, ...
- Trennzeichen: ;, :
- Schlüsselwörter: Goto, If, Then, Halt
- Semantik von WHILE- / LOOP-Programmen
- Turing-vollständig

Beispiele:

Addition $\text{Add}(x, y) = x + y$

```
M1: x0 = x1 + 0;
M2: If x2 = 0 Then Goto M6;
M3: x2 = x2 - 1;
M4: x0 = x0 + 1;
M5: Goto M2;
M6: Halt
```

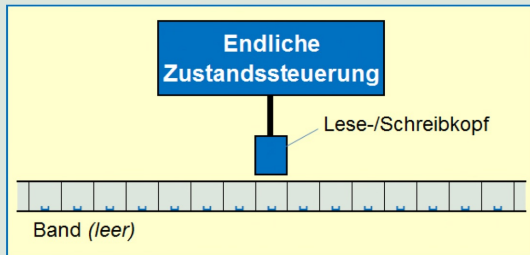
Turingmaschinen (TM)

Ein endlicher Automat ergänzt um:

- Einen Lese-/Schreib-Kopf
- Ein unendliches Band von Zellen

Können mit Lese-/Schreibkopf gelesen und beschrieben werden

Beispiel (Schematische Darstellung einer TM)



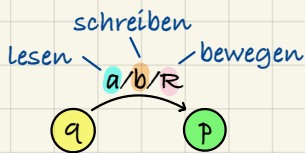
Übergangsfunktion

Partielle Funktion: $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times D$

Bildet 2-Tupel (q, x) auf Tripel (p, y, d) ab:

- $q, p \in Q$ und $x, y \in \Gamma$
- $d =$ Bewegung des Lese-/Schreibkopfes
 - L = Links
 - R = Rechts
 - S = Stabil (keine Bewegung)

z.B. $\delta(q, a) = (p, b, R)$



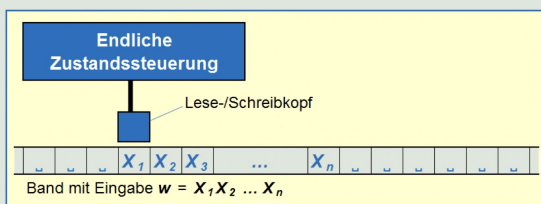
Startkonfiguration

Start auf dem linken Bit

$K = q_0 X_1 X_2 \dots X_n$

q_0 Zustand / X_i Symbole aus Input Alphabet (Σ^*)

Beispiel (Darstellung der Startkonfiguration $K_0 = q_0 X_1 \dots X_n$ der Turing-Maschine M)



Definition (Turing-Maschine)

Eine (deterministische) Turing-Maschine (TM) ist ein 7-Tupel

$$M = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, F) \in \mathcal{EA}$$

mit einer bzw. einem:

- 1 ■ endlichen Menge von Zuständen $Q = \{q_0, q_1, \dots, q_n\} (n \in \mathbb{N})$,
- 2 ■ Eingabealphabet $\Sigma = \{a_1, a_2, \dots, a_m\} (m \in \mathbb{N})$,
 - Übergangsfunktion $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times D, D = \{L, R\}$,
- 3 ■ Startzustand $q_0 \in Q$,
- 4 ■ Menge von akzeptierenden Zuständen $F \subseteq Q$,
 - Bandalphabet Γ (endliche Menge von Symbolen) und $\Sigma \subset \Gamma$ und
 - Leerzeichen $\sqcup$, mit $\sqcup \in \Gamma$ und $\sqcup \notin \Sigma$.

Band

- In einzelne Zellen unterteilt, die beliebiges Symbol aus Γ enthalten
- Beinhaltet zu Beginn die Eingabe (endliches Wort aus Σ^* , alle anderen Zellen enthalten Leerzeichen.)

① Lese-/Schreibkopf kann genau 1 Zelle lesen / schreiben

Konfiguration

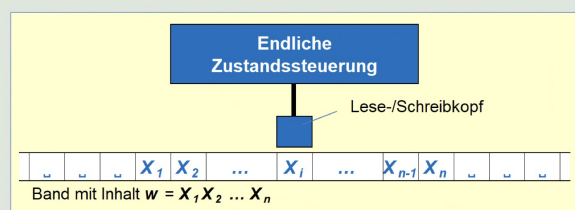
Momentaufnahme einer Berechnung:

- Aktueller Zustand
- Position Lese-/Schreibkopf
- Bandinhalt

Zeichenreihe: $K = X_1 X_2 \dots X_{i-1} q X_i X_{i+1} \dots X_n$

① ist q der Zustand, befindet sich Lese-/Schreibkopf über X_i

Beispiel (Darstellung der Konfiguration $X_1 X_2 \dots X_{i-1} q X_i X_{i+1} \dots X_n$ einer Turing-Maschine M)



Berechnung

Endliche Folge von Berechnungsschritten $K_0 \vdash K_1 \vdash \dots \vdash K_r$

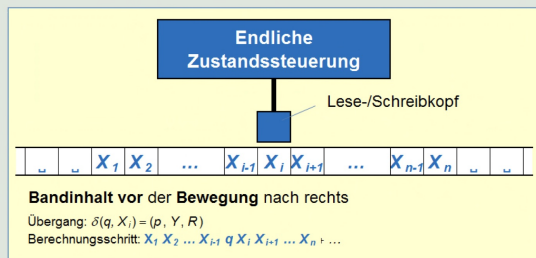
Startkonfiguration /
keine Bewegungen mehr möglich

Berechnungsschritt

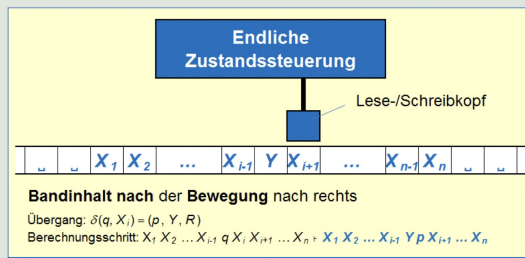
Übergang $\delta(q, X_i) = (p, Y, R)$ mit Bewegung nach rechts:

$X_1 \dots X_{i-1} q X_i X_{i+1} \dots X_n \vdash X_1 \dots X_{i-1} p X_{i+1} \dots X_n$

Beispiel (Bewegung nach rechts einer Turing-Maschine)



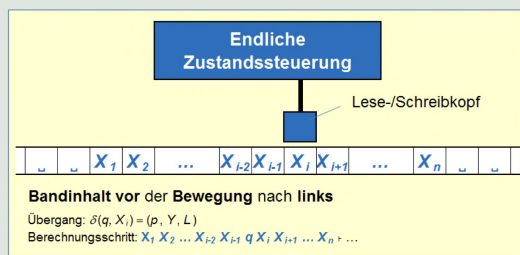
Beispiel (Bewegung nach rechts einer Turing-Maschine)



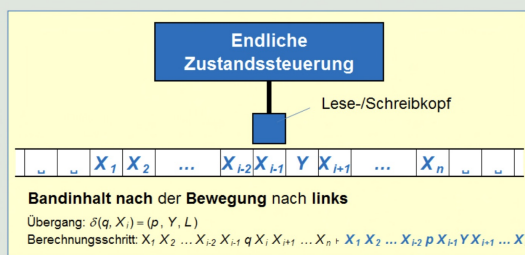
Übergang $\delta(q, X_i) = (p, Y, L)$ mit Bewegung nach links:

$X_1 \dots X_{i-1} q X_i X_{i+1} \dots X_n \vdash X_1 \dots X_{i-2} p X_{i-1} Y X_{i+1} \dots X_n$

Beispiel (Bewegung nach links einer Turing-Maschine)



Beispiel (Bewegung nach links einer Turing-Maschine)



Edge-Cases:

- Leerzeichen nur angeben, wenn für Eindeutigkeit erforderlich (z.B. Links-Bewegung, $i = 1$)
- Leerzeichen links / rechts von Konfigurationsbeschreibung weglassen

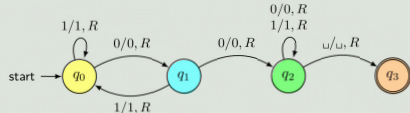
Beispiel (für Berechnungsschritte einer TM mit Bewegungen nach rechts bzw. links am Bandanfang und -ende)

- **Rechts-Bewegung** einer Turing-Maschine mit $\delta(q, X_i) = (p, Y, R)$
 - Für $i = n$ gilt: $X_1 \dots X_{n-1} q X_n \vdash X_1 \dots X_{n-1} Y p \sqcup$
 - Für $i = 1$ und $Y = \sqcup$ gilt: $q X_1 \dots X_n \vdash p X_2 \dots X_n$
- **Links-Bewegung** einer Turing-Maschine mit $\delta(q, X_i) = (p, Y, L)$
 - Für $i = 1$ gilt: $q X_1 \dots X_n \vdash p \sqcup Y X_2 \dots X_n$
 - Für $i = n$ und $Y = \sqcup$ gilt: $X_1 \dots X_{n-1} q X_n \vdash X_1 \dots X_{n-2} p X_{n-1} \sqcup$

Beispiele

Beispiel ($L_1 = \{w_1 00 w_2 \mid w_1, w_2 \in \{0, 1\}^*\}$)

Übergangsdiagramm für M_1 :

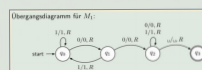


$q_0 101 \vdash 1 q_0 01 \vdash 10 q_1 1 \vdash 101 q_0 \sqcup$
 $\Rightarrow 101 \notin L(M_1)$

Beispiel (Fortsetzung)

Definition für $M_1 = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, \{q_3\})$ mit:

- $Q = \{q_0, q_1, q_2, q_3\}$
- $\Sigma = \{0, 1\}$ und
- $\Gamma = \{0, 1, \sqcup\}$.



Die Übergangsfunktion δ ist gegeben durch:

$\delta(q_0, 0) = (q_1, 0, R), \delta(q_0, 1) = (q_0, 1, R),$
 $\delta(q_1, 0) = (q_2, 0, R), \delta(q_1, 1) = (q_0, 1, R)$ und
 $\delta(q_2, 0) = (q_2, 0, R), \delta(q_2, 1) = (q_2, 1, R), \delta(q_2, \sqcup) = (q_3, \sqcup, R).$

Beispiel (Fortsetzung)

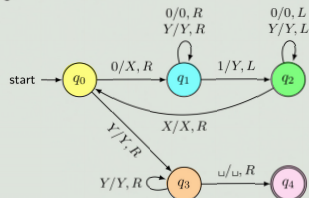
Berechnungen mit M_1 für a) $w = 100$ und b) $w = 010011$:

- a) $q_0 100 \vdash 1 q_0 00 \vdash 10 q_1 0 \vdash 100 q_2 \vdash 100, q_3$
- b) $q_0 010011 \vdash 0 q_1 10011 \vdash 01 q_0 011 \vdash 010 q_1 011 \vdash 0100 q_2 11 \vdash 01001 q_2 1 \vdash 010011 q_3$

Beide Berechnungen enden in q_3 , dem akzeptierenden Zustand ($q_3 \in F$) von M_1 .

Beispiel (Fortsetzung)

Übergangsdiagramm für M_2 :



Beispiel ($L_2 = \{0^n 1^n \mid n \geq 1\}$)

Gegeben sei die Turing-Maschine $M_2 = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, \{q_4\})$ mit:

- $Q = \{q_0, q_1, q_2, q_3, q_4\}$
- $\Sigma = \{0, 1\}$ und
- $\Gamma = \{0, 1, X, Y, \sqcup\}$.

Die Übergangsfunktion δ ist gegeben durch:

$\delta(q_0, 0) = (q_1, X, R), \delta(q_0, Y) = (q_3, Y, R),$
 $\delta(q_1, 0) = (q_1, 0, R), \delta(q_1, 1) = (q_2, Y, L), \delta(q_1, Y) = (q_1, Y, R),$
 $\delta(q_2, 0) = (q_2, 0, L), \delta(q_2, X) = (q_0, X, R), \delta(q_2, Y) = (q_2, Y, L),$
 $\delta(q_3, Y) = (q_3, Y, R)$ und $\delta(q_3, \sqcup) = (q_4, \sqcup, R).$

Beispiel (Fortsetzung)

Berechnungen mit M_2 für a) $w = 01$ und b) $w = 0011$:

- a) $q_0 01 \vdash X q_1 1 \vdash q_2 X Y \vdash X q_0 Y \vdash X Y q_3 \vdash X Y \sqcup q_4$
- b) $q_0 0011 \vdash X q_1 011 \vdash X 0 q_1 11 \vdash X q_2 0 Y 1 \vdash q_2 X 0 Y 1 \vdash X q_0 0 Y 1 \vdash X X q_1 Y 1 \vdash X X Y q_1 1 \vdash X X q_2 Y Y \vdash X q_2 X Y Y \vdash X X q_0 Y Y \vdash X X Y q_3 Y \vdash X X Y Y q_3 \vdash X X Y Y \sqcup q_4$

Beide Berechnungen enden in q_4 , einem akzeptierenden Zustand ($q_4 \in F$) von M_2 .

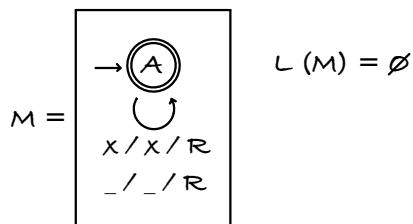
Sprache einer TM

Menge aller Wörter bei der die Berechnung in einem akzeptierenden Zustand endet

Bedingungen:

- TM muss nach endlich vielen Schritten anhalten
- Zustand der letzten Konfiguration muss akzeptierend sein

z.B. TM die nie anhält ($x \in \Sigma$)



Rekursiv aufzählbar

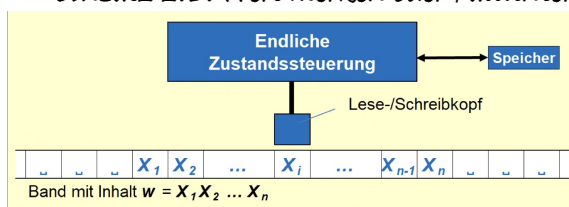
Sprache, die von einer TM akzeptiert wird (alle kontextfreien Sprachen)

Erweiterungen

Lediglich vereinfachte Darstellungen und gleichmächtig wie normale TM

Speicher:

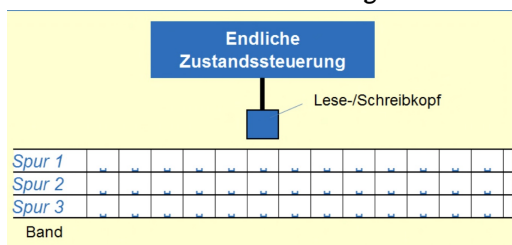
- Speicherung endlich vielen (Daten-) Zustände
- Einsatz z.B. Wert merken oder Fallunterscheidung



Mehrere Spuren:

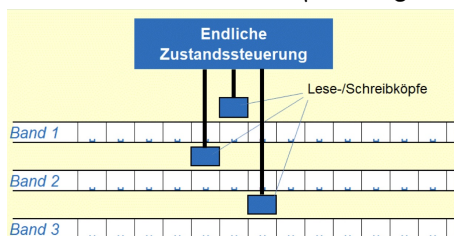
1 Band mit mehreren Spuren (Alle Spuren werden gleichzeitig gelesen)

- Einsatz z.B. Markierungen oder Zwischenergebnisse



Mehrere Bänder:

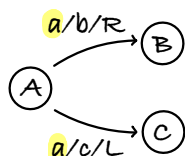
- Lese- / Schreibkopf kann unabhängig auf Band zugreifen
- Einsatz z.B. Vereinfachung von TM (weniger Zustände und Schritte)



Nichtdeterministische TM (NTM)

- Jede Sprache die von NTM akzeptiert wird, wird auch von DTM akzeptiert
- Wie bei NEA werden pro Berechnungsschritt alle Möglichkeiten probiert („Erraten der Lösung“)

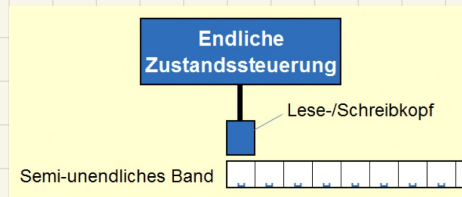
z.B.



Beschränkungen

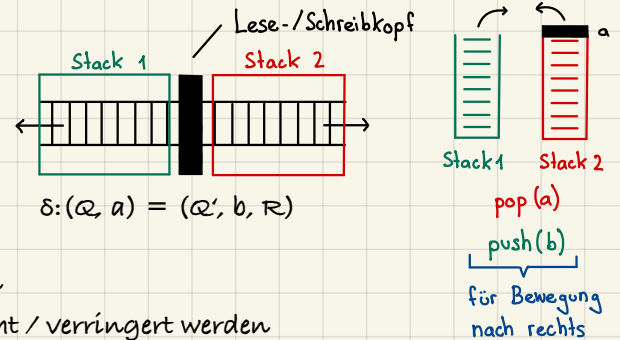
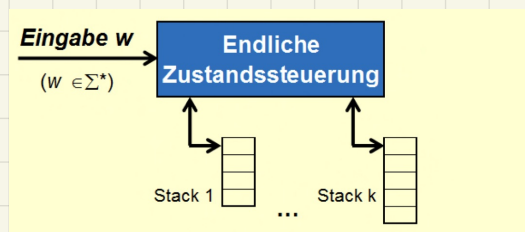
Semi-unendliches Band:

- Nur in eine Richtung unendlich (z.B. nach rechts)
- Links von Anfangsposition sind keine Zellen
- Positionen links von Anfangsposition können nicht eingenommen werden
- Jede Sprache die von TM akzeptiert wird, wird auch von TM mit semi-unendlichem Band akzeptiert



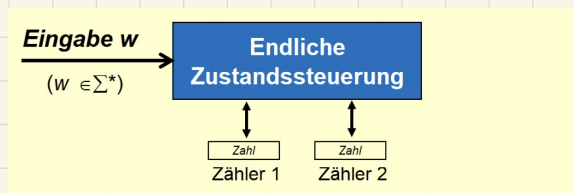
Mehrere Stacks:

- Kellerautomat mit 2 Stacks ist gleichmächtig wie TM
- Jede Sprache die von TM akzeptiert wird, wird auch von 2 Stack Kellerautomat akzeptiert



Zähler-Maschine:

- Zähler kann natürliche Zahl speichern
- Zähler können unabhängig um 1 erhöht / verringert werden
- Unterscheidung nur zwischen Zähler gleich 0 / ungleich 0
- Jede Sprache die von TM akzeptiert wird, wird auch von Zähler-Maschine mit 2 Zählern akzeptiert



Universelle Turingmaschine

- Kann beliebige TM simulieren und somit alle Probleme lösen, die eine TM lösen kann
- Jede TM kann durch eine Binärzeichenreihe dargestellt werden
- Wenn Binärzeichenreihe eine 1 vorrausgestellt wird, kann die Kodierung einer TM als Zahl dargestellt werden

Reguläre Ausdrücke

Regex sind Wörter, die Sprachen beschreiben

Syntax

Σ = beliebiges Alphabet, Sprache RA_{Σ} über Σ

1. $\emptyset, \epsilon \in RA_{\Sigma}$
2. $\Sigma \subset RA_{\Sigma}$

3. $R \in RA_{\Sigma} \Rightarrow (R^*) \in RA_{\Sigma}$
4. $R, S \in RA_{\Sigma} \Rightarrow (RS) \in RA_{\Sigma}$
5. $R, S \in RA_{\Sigma} \Rightarrow (R|S) \in RA_{\Sigma}$

Reguläre Ausdrücke bilden (über $\Sigma = \{a, b\}$):

$\emptyset, \epsilon, a, b, (a^*), (\epsilon^*), ((a^*)b), (((a^*)b)|\emptyset)$

Operatoren

$RA_{\Sigma} = \{\emptyset, \epsilon, *, (,), | \} \cup \Sigma$

Überflüssige Klammern für Lesbarkeit weglassen

- Rangfolge:
1. $*$
 2. Konkatenation
 3. $|$

z.B. $((a(b^*))|c) = ab^*|c$
 $a(b(ba))|b = abba|b$

Erweiterte Syntax

$R^+ = R(R^*)$

$R^? = (R|\epsilon)$

$[R_1, \dots, R_k] = R_1|R_2|\dots|R_k$

z.B. $(ab)^+ = ab(ab)^*$

$a^? = (a|\epsilon)$

$a|b|c = [a, b, c]$

Semantik

Zuordnung eines Regex zu einer Sprache.

- | | | |
|-----------------------------------|---|------------------------------------|
| $L(\emptyset) = \emptyset$ | Leere Sprache | } Grundausdrücke |
| $L(\epsilon) = \{\epsilon\}$ | Sprache, die nur das leere Wort enthält | |
| $L(a) = \{a\}$ für $a \in \Sigma$ | Beschreibt die Sprache $\{a\}$ | |
| $L(R^*) = L(R^+)$ | Kombinierte Wörter von R | } aus bestehenden RA neue RA bauen |

z.B. $\Sigma = \{a, b\}$, $R = a$
 $L(R^*) = L(a^*) = L(a^+) = \{a^*\} = \{\epsilon, a, aa, aaa, \dots\}$

- | | |
|---|---------------------------------------|
| $L(RS) = L(R)L(S)$ | Verkettung von Wörtern (R =prefix) |
| z.B. $R = a^*$, $S = b^*$ | |
| $L(R) = \{\epsilon, a, aa, aaa, \dots\}$ | |
| $L(S) = \{\epsilon, b, bb, bbb, \dots\}$ | |
| $L(RS) = L(R)L(S) = \{\epsilon, b, bb, \dots, a, ab, \dots\}$ | |

- | | |
|--|--|
| $L(R S) = L(R) \cup L(S)$ | Wörter die von R oder S beschrieben werden |
| z.B. $R = a^*$, $S = b^*$ | |
| $L(R S) = L(a^* b^*) = L(a^*) \cup L(b^*)$ | |
| $= \{\epsilon, a, aa, aaa, \dots\} \cup \{\epsilon, b, bb, bbb, \dots\}$ | |
| $= \text{"Folge von a oder Folge von b"}$ | |

Reguläre Sprache

Sprache A über dem Alphabet Σ heisst regulär, falls $A = L(R)$ für $R \in RA_{\Sigma}$ gilt.

- z.B. $R_1 = a^*b^*$: $L(R_1) = \{a^n b^k \mid n \in \mathbb{N}\} = \{b, ab, aab, aaab, \dots\}$
 $R_2 = (aa)^*b^*aba$: $L(R_2) = \{a^n b^k aba \mid n \text{ gerade } k \in \mathbb{N}\} = \{aba, baba, aaaba, aababa, \dots\}$
 $R_3 = (a|ab)^*$: $L(R_3) = \text{"vor jedem b steht ein a"} = \{\epsilon, ab, aa, abab, \dots\}$

Menge der ganzen Zahlen in Dezimaldarstellung : $(-1\epsilon)(11\dots19)(01\dots19)^*|0$

Menge der Binärwörter mit abwechselnd 0 und 1 : $(11\epsilon)(01)^*(01\epsilon)$

Rechenregeln

- | | |
|--------------|----------------|
| $L(R S)$ | $= L(S R)$ |
| $L(R(ST))$ | $= L((RS)T)$ |
| $L(R(SIT))$ | $= L((RIS)IT)$ |
| $L(R(SIT))$ | $= L(RSIRT)$ |
| $L((R^*)^*)$ | $= L(R^*)$ |
| $L(R R)$ | $= L(R)$ |